



Interfaz gráfica y eventos: la lógica no sabe que hay una pantalla

El problema

Hasta ahora, todos los programas arrancaron y terminaron en `main`. Un `main` que arma tres pagos, los mete en una lista, llama a `caja.cerrar(pagos)` e imprime el total. El programa decide qué se ejecuta y en qué orden, y cuando se queda sin instrucciones, termina.

El local quiere otra cosa. Quiere que el cajero tenga una ventana con un campo para escribir el monto, un botón que diga “Registrar” y una línea que muestre el total del día. El cajero escribe, presiona, y el total se actualiza. Y entre un pago y el siguiente pueden pasar dos minutos o dos horas.

La primera versión que se escribe es la más directa. Una sola clase, que es la ventana, atiende el botón y lleva la cuenta:

```
public class VentanaCaja extends JFrame implements ActionListener {

    private final JTextField campoMonto;
    private final JLabel etiquetaTotal;
    private final JButton botonRegistrar;
    private double total;

    public VentanaCaja() {
        super("Caja");
        this.setSize(360, 200);
        this.setResizable(false);
        this.setLayout(null);
        this.campoMonto = new JTextField();
        this.botonRegistrar = new JButton("Registrar");
        this.etiquetaTotal = new JLabel("Total del día: $0.00");
        // ... setBounds() y add() de cada componente ...
        this.botonRegistrar.addActionListener(this);
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        double monto = Double.parseDouble(this.campoMonto.getText());
        this.total += monto;
        this.etiquetaTotal.setText("Total del día: $" + this.total);
    }
}
```

Funciona. Se abre la ventana, el cajero escribe 1500, presiona, y la etiqueta dice 1500. Escribe 800, presiona, y dice 2300.



Caja

Monto: 800

Registrar

Total del dia: \$2300.00

La ventana de la caja después de registrar 1500 y 800.

El costo, como en la clase 8, se ve después. El dueño quiere que ese total sea el mismo que va en el resumen por correo del cierre, y el total está adentro de una ventana. Alguien quiere comprobar que la suma está bien para diez montos, y la única forma es abrir la ventana y hacer diez clicks. Y la caja de la clase 8, la que ya sabía sumar pagos, quedó sin usar, porque la ventana volvió a escribir la suma por su cuenta.

Hay tres cosas mezcladas en esa clase: cómo se ve la pantalla, qué pasa cuando se presiona el botón, y cuánto lleva cobrado el local. La clase 8 dio el nombre del problema. Lo que agrega esta clase es el mecanismo para separarlas, y una regla que ordena todo: la lógica del programa no tiene que saber que hay una pantalla.

Quién tiene el control

Antes de repartir, hay que entender qué cambió respecto de todos los programas anteriores. En `main`, el programa tiene el control. Ejecuta la primera instrucción, después la segunda, entra en un bucle, sale, y termina. El usuario, si participa, lo hace cuando el programa se lo pide, con un Scanner que lo deja esperando en una línea concreta.

Con una ventana, el control se invierte. El programa arma la pantalla, la muestra, y se queda esperando. No sabe si el próximo click va a ser en el botón de registrar o en el de cerrar la ventana, ni cuándo va a llegar. Cada acción del usuario es un evento, y el programa se limita a reaccionar. A esta forma de organizar un programa se la llama programación orientada a eventos, y el flujo ya no lo dicta `main`, lo dicta el usuario.

Hay un detalle que llama la atención la primera vez. El `main` que crea la ventana y la muestra termina enseguida, y el programa sigue corriendo. Eso pasa porque al mostrar la primera ventana, Swing arranca un hilo propio que se queda esperando eventos y llamando al código que corresponda a cada uno. Ese hilo sigue vivo mientras haya una ventana abierta. Con `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`, que es lo habitual, cerrar la ventana termina el programa.

En una interfaz gráfica el programa no ejecuta, espera. El usuario decide qué corre y cuándo.



Los componentes

Java trae dos bibliotecas para hacer ventanas. La primera fue AWT, que usa los componentes del sistema operativo y por eso se ve distinto en cada uno. Sobre ella se construyó Swing, que dibuja sus propios componentes y se ve igual en todos lados. Las clases de Swing viven en el paquete `javax.swing`, y sus nombres empiezan con J.

Para la ventana de la caja alcanza con cuatro:

- `JFrame` es la ventana. Tiene título, tamaño, y adentro van los demás componentes.
- `JLabel` es un texto que el usuario no modifica. Sirve para la etiqueta "Monto" y para mostrar el total.
- `TextField` es un campo de una línea donde el usuario escribe. Lo que escribió se lee con `getText()`, y siempre es un `String`.
- `Button` es el botón. Tiene un texto y, cuando se presiona, avisa.

Armar la ventana es crear los componentes, ubicarlos y agregarlos. Con `setLayout(null)` las posiciones se dan a mano, con `setBounds(x, y, ancho, alto)` en píxeles medidos desde la esquina superior izquierda. Como las posiciones son fijas, la ventana también lo es: `setSize()` le da el tamaño y `setResizable(false)` impide que el usuario lo cambie. Hay formas de ubicar componentes que se acomodan solas cuando la ventana cambia de tamaño, pero para una ventana chica con cuatro cosas adentro las coordenadas fijas alcanzan y no esconden nada.

```
JFrame ventana = new JFrame("Caja");
ventana.setSize(360, 200);
ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
ventana.setResizable(false);
ventana.setLayout(null);

JLabel etiquetaMonto = new JLabel("Monto:");
etiquetaMonto.setBounds(20, 20, 120, 25);
ventana.add(etiquetaMonto);
// ... el campo, el boton y la etiqueta del total, igual ...

ventana.setVisible(true);
```

Hasta la última línea la ventana existe como objeto, pero no está en la pantalla. Es un objeto como cualquier otro, con atributos y métodos, y eso es lo que permite armarla entera antes de mostrarla.

Una ventana es un contenedor con componentes adentro. Hasta `setVisible(true)`, no existe para el usuario.

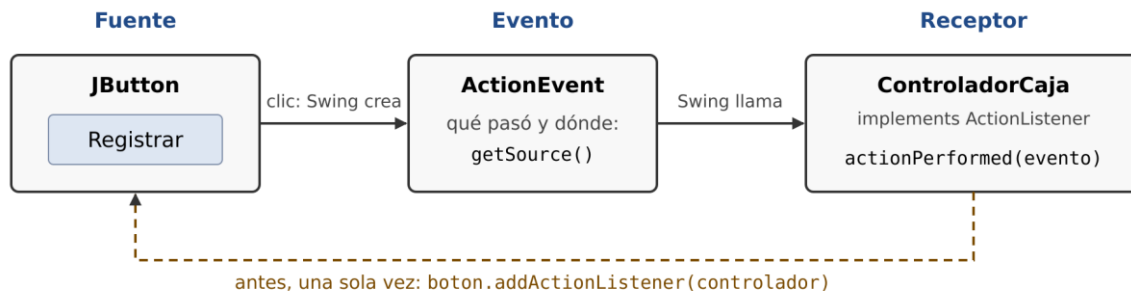
Cómo avisa un botón

El botón no sabe qué tiene que pasar cuando se lo presiona. Sería difícil que lo supiera, porque el mismo `Button` sirve para registrar un pago, para convertir moneda o para cerrar sesión. Lo que el botón sabe es a quién avisar.

Swing resuelve esto con tres piezas. La fuente del evento es el componente donde ocurre la acción, el botón. El evento es un objeto que Swing crea cuando la acción ocurre y que describe



qué pasó, para un botón es un `ActionEvent`. Y el receptor es el objeto que se anotó para que le avisen, y que tiene el código que se ejecuta. A esta forma de repartir el trabajo se la llama delegación de eventos: el botón delega en el receptor la decisión de qué hacer.



Delegación de eventos. El registro se hace una vez; después, cada click recorre el camino de arriba.

Para anotarse, el receptor tiene que cumplir una interfaz. Para los botones es `ActionListener`, que está en el paquete `java.awt.event` y tiene un solo método:

```
public interface ActionListener {  
    void actionPerformed(ActionEvent evento);  
}
```

El receptor la implementa, y se registra en la fuente con `addActionListener()`. Desde ese momento, cada vez que se presione el botón, Swing va a llamar a `actionPerformed()` del receptor, con un `ActionEvent` que dice, entre otras cosas, cuál fue la fuente:

```
public class ControladorCaja implements ActionListener {  
  
    public ControladorCaja(VistaCaja vista, Caja caja) {  
        // ...  
        vista.getBotonRegistrar().addActionListener(this);  
    }  
  
    @Override  
    public void actionPerformed(ActionEvent evento) {  
        // esto corre con cada click  
    }  
}
```

En todo el código del programa no hay una sola línea que llame a `actionPerformed()`. Lo llama Swing, desde su hilo, cuando el usuario hace click. El programa escribe el método y Swing decide cuándo ejecutarlo.

`JButton` es una clase de la biblioteca de Java, escrita hace décadas, que necesita avisar a código que todavía no existe. No puede depender de `ControladorCaja`. Entonces define una interfaz, desde el punto de vista de su propia necesidad, y el código nuevo se adapta a ella. Es *Dependency Inversion* aplicado dentro de la biblioteca, y es lo que hace que un botón sirva para cualquier programa.



El botón no sabe qué hacer, solo a quién avisar. Y a quién avisar lo define una interfaz.

Tres clases en vez de una

Con el mecanismo entendido, se puede volver al problema del principio. `VentanaCaja` mezclaba tres responsabilidades, y cada una la pide alguien distinto. Cómo se ve la pantalla lo pide quien la usa, el cajero. Cuánto lleva cobrado el local lo pide el dueño, y es la misma cuenta que va al cierre. Qué pasa exactamente cuando se presiona el botón es lo que conecta a las otras dos. Tres responsabilidades, tres clases.

El modelo es la clase que sabe del negocio. Para la caja ya existe, es la `Caja` de la clase 8. Aquella recibía la lista completa al cierre; para una pantalla que carga los pagos de a uno le convienen dos métodos más, `registrar(Pago pago)`, que lo suma al día, y `total()`, que devuelve lo acumulado. Sigue sin importar nada de `javax.swing`, y no tiene forma de enterarse de si la están usando desde una ventana, desde un `main` con `println` o desde una prueba. Esa es la propiedad que importa, y es fácil de verificar: basta con mirar los `import`.

La vista es la clase que arma la ventana y sabe mostrar cosas en ella. Tiene el `JFrame` como atributo, no hereda de él, porque una vista no es una ventana, tiene una. Ofrece lo que el controlador necesita y nada más:

```
public class VistaCaja {  
  
    private final JFrame ventana;  
    private final JTextField campoMonto;  
    private final JLabel etiquetaTotal;  
    private final JButton botonRegistrar;  
  
    public VistaCaja() {  
        // ... arma la ventana y los cuatro componentes ...  
    }  
  
    public void mostrar() {  
        this.ventana.setVisible(true);  
    }  
  
    public JButton getBotonRegistrar() {  
        return this.botonRegistrar;  
    }  
  
    public String getTextoMonto() {  
        return this.campoMonto.getText();  
    }  
  
    public void mostrarTotal(double total) {  
        this.etiquetaTotal.setText(String.format("Total del día: $%.2f", total));  
    }  
  
    public void mostrarError(String mensaje) {  
        JOptionPane.showMessageDialog(this.ventana, mensaje, "Error",  
JOptionPane.ERROR_MESSAGE);  
    }  
}
```



El botón se expone porque el controlador tiene que registrarse en la fuente del evento. El campo y la etiqueta quedan adentro: el controlador no necesita saber que el total se muestra en un `JLabel`, le alcanza con pedir que se muestre. `JOptionPane` es un cartel emergente con un mensaje y un botón de aceptar, y va en la vista por la misma razón.

El controlador es la clase que escucha. Implementa `ActionListener`, recibe la vista y el modelo por constructor, y en `actionPerformed()` traduce en las dos direcciones: lo que hay en la pantalla se convierte en una llamada al modelo, y lo que el modelo devuelve se convierte en algo que la vista muestre:

```
public class ControladorCaja implements ActionListener {

    private final VistaCaja vista;
    private final Caja caja;

    public ControladorCaja(VistaCaja vista, Caja caja) {
        if (vista == null || caja == null) {
            throw new IllegalArgumentException("La vista y la caja son obligatorias");
        }
        this.vista = vista;
        this.caja = caja;
        this.vista.getBotonRegistrar().addActionListener(this);
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        try {
            double monto = Double.parseDouble(this.vista.getTextoMonto());
            this.caja.registrar(new PagoEfectivo(monto));
            this.vista.mostrarTotal(this.caja.total());
        } catch (NumberFormatException ex) {
            this.vista.mostrarError("Ingrese un numero");
        } catch (IllegalArgumentException ex) {
            this.vista.mostrarError(ex.getMessage());
        }
    }
}
```

Dos cosas nuevas en ese método. Lo que el usuario escribió es un `String`, y `Double.parseDouble()` lo convierte en número. Si el texto no es un número, lanza `NumberFormatException`, de la misma familia que la `IllegalArgumentException` que los constructores vienen lanzando desde la clase 3. El bloque `try` intenta ejecutar las tres líneas, y si alguna lanza, el `catch` que corresponda atrapa el error y sigue desde ahí. Es la primera vez que en el curso se atrapa una excepción en vez de solo lanzarla, y por ahora alcanza con esto: sin el `try`, un texto mal escrito termina en una excepción que `Swing` imprime en la consola, y el usuario no ve nada; con él, ve un cartel y vuelve a intentar. El segundo `catch` atrapa lo que rechaza el propio modelo, un monto negativo por ejemplo, y muestra el mensaje que el constructor de `PagoEfectivo` ya tenía.

Queda `main`, que es el único lugar donde aparecen los `new` de las tres clases:



```
public static void main(String[] args) {  
    Caja caja = new Caja(new ServicioCorreo("smtp.lapampa.com", 587));  
    VistaCaja vista = new VistaCaja();  
    ControladorCaja controlador = new ControladorCaja(vista, caja);  
    vista.mostrar();  
}
```

Es exactamente la inyección de dependencias de la clase 8. El controlador no construye ni la vista ni la caja, las recibe. Y la caja es la misma de la clase pasada, con su notificador y todo.

El modelo no importa javax.swing. Esa es la prueba de que la lógica no sabe que hay una pantalla.

Una carpeta por responsabilidad

Hasta ahora todas las clases del curso vivieron juntas en `src`. Con tres responsabilidades separadas conviene que la separación se vea también en las carpetas, y Java tiene un mecanismo para eso, los paquetes. Un paquete es una carpeta con nombre, y las clases que están adentro lo declaran en su primera línea:

```
src/  
  modelo/  
    Caja.java           package modelo;  
    Pago.java           package modelo;  
    PagoEfectivo.java   package modelo;  
  vista/  
    VistaCaja.java      package vista;  
  controlador/  
    ControladorCaja.java package controlador;  
  Programa.java         sin package, en la raíz
```

El nombre de la carpeta y el del paquete tienen que coincidir, y ese es el error más común la primera vez. Una clase que necesita a otra de un paquete distinto la importa, igual que se importa `JFrame` de `javax.swing`:

```
package controlador;  
  
import modelo.Caja;  
import modelo.PagoEfectivo;  
import vista.VistaCaja;  
  
public class ControladorCaja implements ActionListener {  
    // ...  
}
```

Hay un detalle que hasta hoy no importaba. Una clase sin `public` adelante solo se ve desde su propio paquete. Como `caja` la usa el controlador, que está en otro, tiene que ser `public class caja`. Es el mismo criterio del encapsulamiento de la clase 3, aplicado a clases enteras en vez de a atributos.

Lo que se gana es que la regla de esta clase queda a la vista. Los tres archivos de `modelo` no tienen un solo `import` de `vista`, de `controlador` ni de `javax.swing`. El controlador importa a los



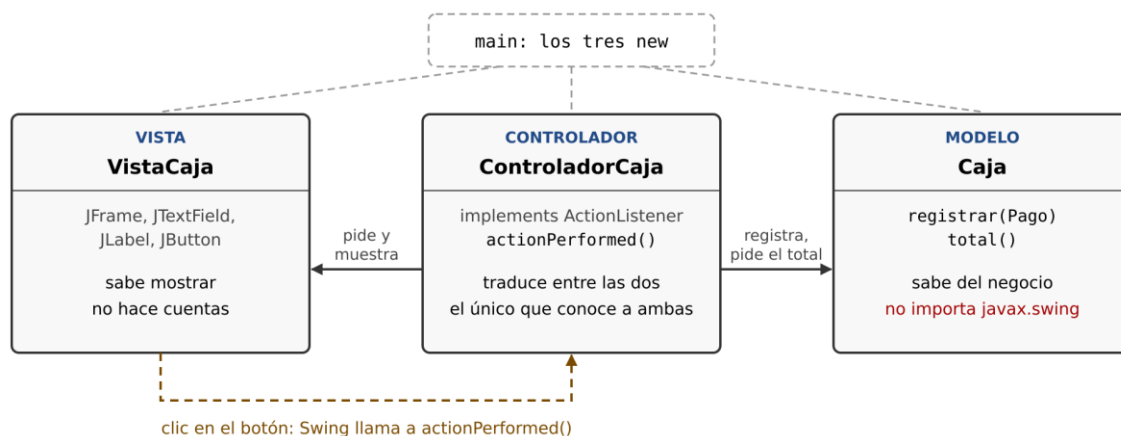
otros dos, porque es el único que los conoce. Y `Programa`, en la raíz, importa lo que necesita para armar los tres.

El paquete `modelo` no importa vista ni controlador. Se verifica leyendo tres líneas.

Qué se ganó

Con las tres clases separadas, los tres pedidos del principio tienen un lugar. El total del resumen por correo sale de `caja.total()`, el mismo objeto que usa la ventana. Comprobar la suma para diez montos es un `main` de diez líneas que llama a `registrar()` y no abre ninguna ventana. Y si el cajero pide que el total se vea más grande, o que el botón esté a la derecha, se toca `VistaCaja` y nada más.

A esta manera de repartir se la conoce como Modelo-Vista-Controlador, y aparece con variantes en casi todas las bibliotecas de interfaz gráfica y en la mayoría de los frameworks web. Las variantes cambian qué hace exactamente el controlador y cómo se entera la vista de que el modelo cambió, pero la idea de fondo es la de esta clase: el modelo no sabe que hay una pantalla, la vista no sabe qué significa lo que muestra, y el controlador es el único que conoce a las dos.



Las tres clases y quién conoce a quién. Las flechas punteadas de arriba son los `new` de `main`.

Tres preguntas alcanzan para saber si el reparto está bien hecho. ¿El paquete `modelo` importa algo de `javax.swing`, de `vista` o de `controlador`? No debería. ¿La vista hace alguna cuenta o toma alguna decisión del negocio? No debería. ¿Se puede probar el modelo desde un `main` sin abrir una ventana? Debería.

Bibliografía sugerida para profundizar

Aguilar, L. J. (2011). *Programación en Java: algoritmos, programación orientada a objetos e interfaz gráfica de usuario*. McGraw-Hill.



- Capítulo 20, “Gráficos I. GUI/Swing”, para los componentes, y capítulo 21, “Gráficos II. Componentes y eventos”, para el modelo de delegación. En castellano y con los mismos ejemplos de esta clase.

Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects*.

- Capítulo 26, “Swing GUI Components: Part 1”. Explica JFrame, JLabel, JTextField, JButton y el manejo de eventos con ActionListener, en ese orden.

Belmonte Fernández, O. (2012). *Desarrollo de proyectos informáticos con tecnología Java*. Universitat Jaume I.

- Capítulo 11, “Interfaces gráficas de usuario”, en especial 11.4, “Detección de eventos: escuchadores”. Es gratuito y breve.

Martin, R. C. (2017). *Clean Architecture*. Prentice Hall.

- Capítulo 22, “The Clean Architecture”, para ver que la regla de esta clase, la lógica no sabe que hay una pantalla, es la misma que ordena sistemas mucho más grandes.